Documentación de la migración de JavaScript a TypeScript en la aplicación web *Finanzas del hogar*

Juan Carlos Merchan
Universidad — Electiva 2 – Desarrollo Web
09/11/2026

***Resumen*—Se documenta la migración de una aplicación web monolítica de gestión financiera del hogar, construida con Express, EJS y MongoDB, desde JavaScript con módulos ECMAScript hacia TypeScript estricto. El proceso conservó el comportamiento funcional y adoptó programación orientada a objetos en los modelos de datos, mediante interfaces y clases. El esquema de verificación combinó el compilador de TypeScript con pruebas funcionales de extremo a extremo: el árbol migrado pasó de 686 a 0 errores de `tsc --noEmit` y superó 204 de 204 comprobaciones funcionales. El artículo describe el contexto del sistema original, la metodología por capas, la arquitectura resultante, cinco diagramas del Lenguaje Unificado de Modelado (UML) y las lecciones aprendidas. Su enfoque es documental: no se rehace la migración ni se propone una reingeniería.**

***Palabras clave*—migración de software, TypeScript, Express, Mongoose, UML, verificación**

I. Introducción

I-A. Contexto y problema

La aplicación web *Finanzas del hogar* es un proyecto académico de la asignatura Electiva 2 – Desarrollo Web. Permite administrar hogares, cuentas, transacciones de ingreso y gasto, categorías, presupuestos y reportes. El sistema se implementó originalmente como un monolito JavaScript ESM sobre Express, con vistas EJS y persistencia en MongoDB mediante Mongoose. Aunque el código funcionaba, la ausencia de tipos hacía que los contratos entre capas existieran solo en la documentación y en la memoria del autor: los cuerpos de petición llegaban como objetos sin forma declarada, los servicios aceptaban y devolvían estructuras verificadas únicamente en tiempo de ejecución y cualquier refactorización exigía recorrer manualmente los puntos de uso. El problema documentado no fue un fallo funcional, sino un costo estructural de mantenimiento y una barrera para demostrar corrección.[1]

I-B. Objetivo y alcance

El objetivo es documentar, con evidencia verificable, la migración de ese monolito a TypeScript estricto, sin rehacerla y sin alterar el comportamiento observable. El alcance comprende el backend de 53 archivos, las dos fuentes del frontend y los artefactos generados que se publican con el sitio. Quedan fuera del proceso migratorio las funcionalidades incorporadas después del cierre de la migración, que se describen como estado actual en la Sección IV y se delimitan en la Sección VI.

I-C. Aportes

El artículo ofrece tres aportes. Primero, una descripción ordenada de la migración por capas, con las decisiones que permitieron pasar de un árbol sin tipos a uno estrictamente verificado. Segundo, la formalización del patrón de programación orientada a objetos aplicado a los modelos de Mongoose, incluida la distinción entre interfaz de contrato y clase de estáticos. Tercero, un protocolo de verificación reproducible (686 a 0 errores, 204/204 comprobaciones) y trazabilidad de snapshots por commit (`fb1ad1d` a `34016bd`).

I-D. Organización

La Sección II describe el sistema original en JavaScript. La Sección III presenta la metodología y el proceso de migración. La Sección IV caracteriza la arquitectura del sistema migrado en su estado actual. La Sección V explica los cinco diagramas UML. La Sección VI reúne resultados, validación y lecciones aprendidas. La Sección VII formula las conclusiones y la Sección VIII lista las referencias.

II. Contexto del sistema original en JavaScript

II-A. Descripción funcional

El sistema organiza la información financiera por hogar. Cada hogar agrupa miembros con rol de administrador o miembro, y delimita cuentas, categorías de ingreso y gasto, transacciones, presupuestos mensuales y reportes. La autenticación se resuelve con sesiones basadas en JSON Web Tokens almacenados en cookies, y la documentación de la interfaz de programación se publica con Swagger. La aplicación expone dos superficies: rutas web que renderizan vistas EJS y rutas de interfaz de programación que responden JSON.

[1] La plantilla IEEEtran fue adaptada localmente para usar Computer Modern en lugar de Times.

Tabla I: Stack declarado al momento de la migración (commit `34016bd`)

Componente	Tecnología	Versión	Evidencia
Servidor HTTP	Express	^5.2.1	`package.json`
Vistas	EJS	^7.0.1	`package.json`
Persistencia	Mongoose	^9.11.0	`package.json`
Autenticación	jsonwebtoken	^9.0.3	`package.json`
Documentación	swagger-jsdoc	^6.3.0	`package.json`
Configuración	dotenv	^18.0.5	`package.json`
Runtime	Node.js ESM	no disponible	`package.json`
Ejecución TS	tsx	^4.23.15	`package.json`
Compilador	TypeScript	^7.0.2	`package.json`
Bundler	esbuild	^0.28.2	`package.json`

II-B. Stack tecnológico original

La Tabla I resume las tecnologías declaradas en el momento de la migración. Las versiones corresponden a los rangos del archivo `package.json` en el commit `34016bd`; no se presentan versiones instaladas ni el estado actual de las dependencias.

II-C. Organización del código

El backend se organizaba en carpetas técnicas: `config`, `controllers`, `driver`, `middlewares`, `models`, `routes`, `services`, `types`, `utils` y `views`. Los controladores se dividían en web y de interfaz de programación; los servicios concentraban las reglas de negocio y los modelos la definición de esquemas. La migración abarcó 53 archivos del backend y dos fuentes del frontend, sin introducir una nueva capa arquitectónica.

II-D. Limitaciones del JavaScript sin tipos

La ausencia de anotaciones no impedía operar el sistema, pero trasladaba la verificación al tiempo de ejecución. Los errores de forma en un cuerpo de petición se descubrían al responder una solicitud, no al compilar. Los contratos de los servicios dependían de convenciones y los cambios de firma podían propagarse sin aviso hasta un punto de uso lejano. A ello se sumaba la documentación OpenAPI [1], que describía la interfaz pero no obligaba al código a respetarla.

II-E. Estado de partida

El punto de partida de la migración se fijó con la cifra de 686 errores de `tsc --noEmit` sobre el árbol recién renombrado. Es decir, no se trata de 686 defectos funcionales, sino del total de diagnósticos emitidos por el compilador al activar la verificación estricta sobre los archivos que antes se ejecutaban como JavaScript. Ese valor operó como línea base y como criterio de avance: cada capa se consideraba terminada cuando reducía el conteo sin introducir cambios de comportamiento. El estado inicial corresponde al commit `fb1ad1d` (Anexo A.2).

III. Metodología y proceso de migración

III-A. Estrategia general

La migración se planteó como una transformación de tipos, no como una reescritura. Se preservaron las rutas, los códigos HTTP, los textos de respuesta y los flujos de error. La verificación se separó de la ejecución: el compilador se ejecuta con `tsc --noEmit` y el servidor se inicia con `tsx` [2], que transpila en caliente sin generar un directorio de distribución. Esta separación evitó un paso de compilación en el despliegue y permitió tratar el tipado como una propiedad verificable del repositorio.

III-B. Renombrado mecánico

La primera fase fue deliberadamente mecánica. Se renombraron 53 archivos `.mjs` a `.ts` conservando el historial de git, se sustituyeron 106 especificadores de importación para que apuntaran a la nueva extensión y se actualizaron los patrones de `swagger-jsdoc` que alimentan el documento OpenAPI. La fase se cerró con dos comprobaciones negativas: ninguna extensión `.mjs` permanecía en `src/` y ninguna referencia a `.mjs` sobrevivía en el código.

III-C. Configuración del compilador

La configuración adoptada se reproduce en el Listado 1.[2] Su propósito es demostrar que la verificación estricta se aplicó a todo el árbol, incluido el frontend, y que el compilador se usó sin emitir archivos.

Listado 1: Configuración del compilador (`tsconfig.json`)

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "types": ["node"],
    "strict": true,
    "noEmit": true,
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "isolatedModules": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  },
  "include": ["src"]
}
```

Tres decisiones explican el resultado. `strict` activa el conjunto completo de comprobaciones [3], [4]; `verbatimModuleSyntax` obliga a declarar con `import type` los imports que solo aportan tipos; y `allowImportingTsExtensions` permite que los imports internos lleven la extensión `.ts`, coherente con el modo ESM y con `tsx`. La opción `lib` incluye el DOM porque las fuentes del frontend viven bajo `src/` y se validan en la misma ejecución. La configuración completa corresponde al estado del archivo en el commit `34016bd`.

[2]El fragmento se reproduce del estado del archivo en el commit `34016bd`.

III-D. Orden de migración por capas

La migración avanzó de abajo hacia arriba, con el compilador como puerta de cada etapa: configuración, utilidades y driver; middlewares; modelos; servicios; controladores y, finalmente, el punto de entrada. El orden no fue arbitrario: cada capa dependía de contratos ya estabilizados en la anterior, de modo que los errores restantes señalaban deuda localizada y no ruido cruzado. Los servicios recibieron los cuerpos de petición como `unknown` y los transformaron en estructuras limpias antes de persistir; los controladores se anotaron como manejadores de Express; y el punto de entrada se tipó junto con la corrección de una doble invocación de arranque.

III-E. Programación orientada a objetos en los modelos

El hallazgo técnico más relevante fue que `Model` de Mongoose se declara como interfaz, no como clase. Extender una clase desde esa interfaz colapsa los miembros heredados a `any` y anula el chequeo. El patrón adoptado separa tres piezas: la interfaz del documento, la interfaz del contrato del modelo y una clase que implementa los estáticos y se copia al esquema con `loadClass` [5]. El Listado 2 reproduce la aplicación del patrón al modelo de usuario.

Listado 2: Contrato de modelo y estáticos copiados con `loadClass`

```
export interface IUserModel extends Model<IUser> {
  findByEmail(email: string): Promise<IUser | null>;
  findLeanById(userId: Types.ObjectId | string): Promise<IUser |
      null>;
}

class UserModel extends Model<IUser> {
  static findByEmail(this: IUserModel, email: string) {
    return this.findOne({ email: email.trim().toLowerCase() }).lean
      ();
  }
}

userSchema.loadClass(UserModel);
export const User = mongoose.model<IUser, IUserModel>('User',
      userSchema);
```

El patrón se aplicó a siete modelos y se acompañó de enumeraciones con valores idénticos a los almacenados, de modo que la validación de los esquemas no cambiara. Los estáticos de dominio, como la búsqueda por correo o la verificación de pertenencia al hogar, quedaron expuestos en el contrato y los servicios los consumieron en lugar de construir filtros manuales. El fragmento se reproduce del estado del archivo en el commit `34016bd`.

III-F. Contratos de petición y augmentación de Express

Las propiedades que los middlewares inyectan en la petición se declararon mediante augmentación de tipos, como muestra el Listado 3.[3] Esto convierte el acceso a `req.home` o `req.user` en una operación verificada y obliga a resolver la posibilidad de que no existan.

[3]El fragmento se reproduce del estado del archivo en el commit `34016bd`.

Listado 3: Augmentación de `Express.Request`

```
declare global {
  namespace Express {
    interface Request {
      user?: IUser;
      home?: IHome;
      homeId?: string;
      homeRole?: Role;
    }
  }
}
```

En las rutas protegidas, los middlewares garantizan la presencia de esas propiedades; por eso se empleó la aserción de no nulidad en lugar de añadir guardas que habrían alterado los flujos de error. La aserción es una operación de tipo y no modifica el valor en tiempo de ejecución. La declaración completa corresponde al commit `34016bd`; su uso se describe en el Anexo A.2.

III-G. Errores y conversiones de tipo

El compilador tipa el parámetro de `catch` como `unknown`. Para no perder el chequeo estricto se introdujo un guardia `isHttpError` y se reescribieron 31 condiciones de estado, de modo que cada rama estrecha el tipo antes de leer el mensaje o los detalles. Las conversiones se limitaron a operaciones de tipo: nunca transforman el valor en tiempo de ejecución. Entre los ajustes puntuales figuran el paso de `req.params` a cadenas tipadas, la aritmética de fechas mediante `getTime` y el tratamiento de cargas JWT que pueden ser cadenas. Con ello se corrigieron dos riesgos latentes sin cambiar respuestas: el acceso a propiedades de una cadena y la aritmética implícita entre fechas.

III-H. Frontend en TypeScript

Las dos fuentes del frontend se trasladaron a `src/frontend` y se compilaron con esbuild hacia `public/js` [6]. Los artefactos generados se conservan en el repositorio porque el despliegue no ejecuta un paso de compilación. Se mantuvieron las funciones autoejecutables, de modo que el bundle no introduce módulos ni imports en tiempo de ejecución. La validación de formularios y el gráfico del panel usan tipos del DOM y un contrato mínimo para la biblioteca de gráficos, sin dependencias adicionales.

III-I. Verificación

La Tabla II reúne exclusivamente las cifras del momento migratorio. El typecheck pasó de 686 a 0 errores y las suites HTTP quedaron en 204 comprobaciones exitosas sobre 204. Las cifras actuales del repositorio no se incluyen en esta tabla porque corresponden a etapas posteriores.

Además del conteo, se comprobaron regresiones puntuales: la documentación OpenAPI respondió con el documento generado y sus recursos, las rutas de archivos estáticos sirvieron los bundles del frontend y los mensajes de sesión se conservaron. El criterio de cierre fue doble: compilador sin errores y suites sin fallos. Esos dos conjuntos de comprobaciones, ejecutados sobre el commit `34016bd`, son la fuente de la Tabla II.

Tabla II: Verificación al cierre de la migración (commit `34016bd`)

Suite	Comprobaciones	Estado
Estado base (`setup`)	—	correcto
Transacciones (`i4`)	52	52/52
Presupuestos y permisos (`i5`)	62	62/62
Reportes y recursos (`i6`)	49	49/49
Autenticación y API (`i7smoke`)	41	41/41
Total	204	204/204

Fuente: suites HTTP ejecutadas sobre el commit `34016bd` (05/11/2026).

Tabla III: Dimensiones del repositorio en el estado actual (commit `cebc62a`, 08/11/2026)

Categoría	Cantidad	Fuente
Archivos TypeScript en `src/`	67	`git ls-files`
Vistas EJS	24	`src/views`
Modelos Mongoose	9	`src/models`
Controladores	21	`src/controllers`
Servicios	11	`src/services`
Archivos de rutas	11	`src/routes`

Fuente: repositorio en `cebc62a` (HEAD al 08/11/2026).

III-J. Decisiones y alternativas descartadas

Las alternativas evaluadas y su motivo de descarte se resumen en el Anexo A.1; ninguna de ellas forma parte del flujo final.

III-K. Trazabilidad de snapshots

Los snapshots se extrajeron de dos commits: `fb1ad1d`, que fija el estado previo a la migración, y `34016bd` ("Migración completa a TypeScript estricto"), que cierra la migración. El estado actual del árbol puede diferir de esos snapshots por las fases posteriores A–E, que este artículo no documenta como parte del proceso migratorio. La comparación entre ambos commits se resume en el Anexo A.2.

IV. Arquitectura y diseño del sistema migrado

Esta sección describe la arquitectura en el estado actual del repositorio y no en el instante de cierre de la migración. Esa distinción es necesaria porque las fases posteriores añadieron funcionalidad sobre la base ya tipada, sin alterar la organización por capas que dejó el proceso migratorio.

IV-A. Estado actual y métricas

El estado actual se fijó en el commit `cebc62a` (08/11/2026), posterior al cierre migratorio de `34016bd` (05/11/2026). La Tabla III resume el tamaño del árbol y declara la fuente de cada cifra, de modo que las métricas puedan reconstruirse desde el historial. Las cifras de esta tabla describen el repositorio completo y no se comparan con las de la Tabla I, que consigna el stack declarado en el commit de cierre de la migración.

IV-B. Capas y flujo de una petición

El punto de entrada es `src/app.ts`. Allí se configura el motor de vistas EJS, el middleware de diseño, los analizadores de cuerpo, la lectura de cookies, los archivos estáticos y la autenticación. Después se registra un middleware que carga en `res.locals` los datos comunes de la navegación y, por último, se montan las superficies de la aplicación. El orden importa: las rutas de interfaz de programación y de salud se resuelven antes del middleware de locales para no ejecutar consultas innecesarias, y los manejadores de error se registran al final.

Una petición recorre cinco puntos: ruta, middleware, controlador, servicio y modelo. La ruta declara el verbo y la ruta; el middleware valida sesión, hogar y rol; el controlador traduce la petición a una llamada de servicio; el servicio aplica las reglas de negocio y consulta los modelos; y el modelo persiste en MongoDB a través del controlador de base de datos. Las respuestas web renderizan una vista EJS y las respuestas de interfaz de programación devuelven JSON. Los errores ascienden por la cadena y los centraliza el middleware de error, que distingue entre `HttpError` y fallos no previstos.

IV-C. Configuración y arranque

La configuración se concentra en `src/config`. El módulo de entorno lee y valida las variables necesarias; el de base de datos establece la conexión; el de Swagger compone el documento OpenAPI; y el de monedas fija los símbolos y la precisión. El arranque se encapsula en una función asíncrona que primero conecta con MongoDB y solo después abre el puerto; si la conexión falla, el proceso termina con un mensaje explícito. Cuando la variable de entorno de pruebas está activa, el arranque no se ejecuta y el módulo exporta la aplicación para que las suites la consuman directamente [7].

IV-D. Rutas y controladores

Las rutas se dividen en dos árboles. El árbol de interfaz de programación monta los recursos de autenticación, hogares, invitaciones y notificaciones; el árbol web añade las mismas familias en formato de vistas y una ruta de presentación. Los 21 controladores se reparten entre la superficie JSON y la superficie EJS. Los controladores son deliberadamente delgados: extraen parámetros, delegan en un servicio y eligen el código de estado o la vista. Esta delgadez es consecuencia de la migración, porque permitió tipar los manejadores con las firmas de Express sin arrastrar lógica de negocio hacia la capa HTTP.

IV-E. Servicios y reglas de negocio

Los 11 servicios concentran las reglas que antes estaban dispersas. Cada servicio expone funciones con entradas tipadas, valida las estructuras antes de persistir y devuelve representaciones públicas, no documentos crudos. Esa frontera evita que los controladores conozcan detalles del esquema y que las vistas reciban campos internos. El

Tabla IV: Modelos con contrato de programación orientada a objetos, agrupados por dominio

Dominio	Modelos	Estáticos representativos
Financiero	Cuenta, Categoría, Transacción, Presupuesto	búsqueda por hogar y por mes
Usuarios y hogar	Usuario, Hogar, Invitación	búsqueda por correo y pertenencia
Soporte	Meta, Notificación	listado por usuario y conteo de no leídas

Fuente: `src/models` en el commit `cebc62a` (08/11/2026); los nueve archivos invocan `loadClass`.

servicio de transacciones, por ejemplo, valida las referencias de cuenta y categoría, crea el movimiento y dispara la evaluación de presupuestos; el de reportes agrega por rango de fechas y por categoría; y el de hogares resuelve la pertenencia y el rol antes de permitir cualquier operación.

IV-F. Modelos y contratos de programación orientada a objetos

Los nueve modelos aplican el mismo contrato: una interfaz de documento, una interfaz de modelo que extiende `Model` y una clase con los estáticos que se copian mediante `loadClass`. La Tabla IV los agrupa por dominio para mostrar que el patrón se generalizó y no quedó reducido al modelo de usuario.

La tabla reúne los nueve modelos (4 + 3 + 2), de modo que el conteo pueda verificarse sin recorrer el árbol de archivos. La agrupación también explica la dependencia entre dominios: las entidades financieras se delimitan por el hogar, las de usuarios y hogar resuelven la membresía, y las de soporte consumen referencias de ambos. El detalle de los contratos se amplía en la Sección V-E, donde se presentan los diagramas de clases.

IV-G. Middlewares, tipos y utilidades

Los cuatro middlewares cubren autenticación, resolución de hogar, verificación de rol y manejo de errores. La augmentación de `Express.Request` declarada en `src/types/express.d.ts` es el contrato que enlaza esos middlewares con los controladores. Las utilidades se limitan a dos responsabilidades: el formato monetario y la cookie del hogar activo. El controlador de MongoDB encapsula la conexión y no expone el cliente a las capas superiores. Esta separación mantiene el árbol pequeño y hace que cada archivo tenga una razón clara para existir.

IV-H. Vistas y frontend

Las 24 vistas EJS se organizan por recurso, con un diseño base y parciales para navegación y mensajes. El panel, las cuentas, los presupuestos, las categorías, las transacciones, las metas, las notificaciones, los hogares y las invitaciones cuentan con sus propias pantallas. El frontend se compila con esbuild a partir de dos fuentes TypeScript y se publica como dos paquetes en `public/js`; el despliegue no ejecuta un paso de compilación, por lo que los artefactos se conservan en el repositorio [6].

IV-I. Evolución posterior: metas y notificaciones

Las fases A–E incorporaron metas y notificaciones con posterioridad al cierre migratorio; este artículo las describe en la arquitectura actual (IV) pero no las contabiliza en los resultados migratorios (VI.1). Las metas se modelan con aportes embebidos y progreso derivado, de modo que el ahorro no se almacena como un campo duplicado. Las notificaciones se generan cuando un gasto excede el presupuesto mensual de su categoría, se deduplican por usuario, presupuesto y mes, y se resuelven automáticamente cuando el exceso desaparece. Ambas funcionalidades reutilizan los contratos de modelo y los servicios ya tipados, lo que confirma que la base migrada admitió extensiones sin rehacer la arquitectura.

V. Diagramas UML

Los diagramas se elaboraron con PlantUML y se interpretan conforme a UML 2.5.1 [8], [9]. Se presentan cinco tipos de diagrama en siete figuras: dos de casos de uso, uno de componentes, uno de secuencia, uno de colaboración y dos de clases. Las figuras se discuten en el texto con una presentación y un análisis, y se incluyen como material vectorial en el documento final.

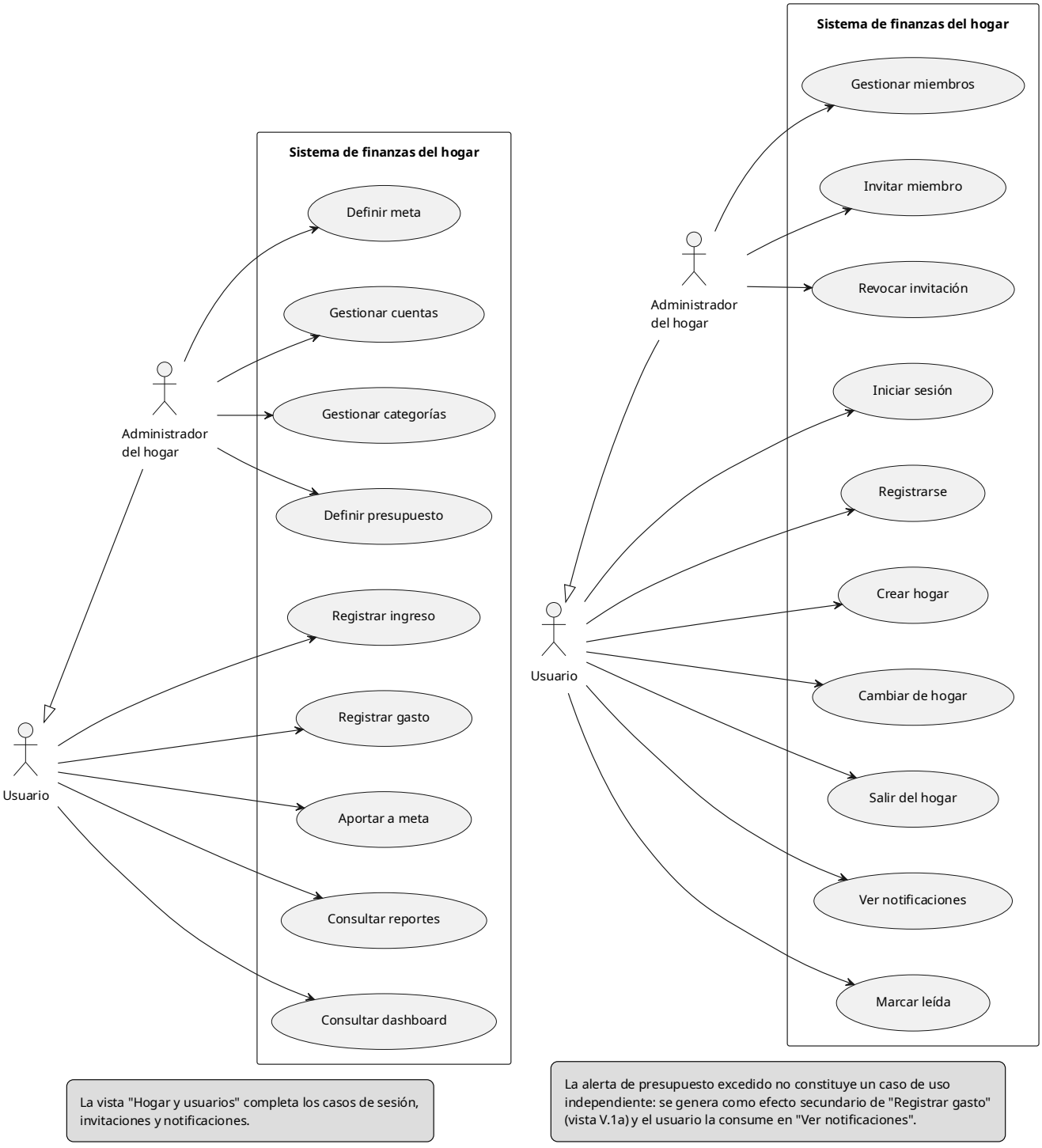
V-A. Diagramas de casos de uso

V-A1. Operaciones financieras: La Fig. 1a presenta las operaciones que un miembro del hogar realiza sobre cuentas, categorías, transacciones y presupuestos. Cada caso de uso corresponde a una acción iniciada por el usuario y delimitada por el hogar activo. La Fig. 1a evidencia que la autenticación y la selección de hogar son condiciones previas comunes, no casos de uso independientes, y que las operaciones de consulta y de escritura comparten los mismos actores.

V-A2. Hogar y usuarios: La Fig. 1b agrupa las operaciones de registro, autenticación, gestión de hogares, miembros e invitaciones, junto con la consulta de notificaciones. Se descartó un actor “Sistema” porque los casos identificados son iniciados por usuarios o por tareas explícitas (envío de invitaciones), no por procesos autónomos; la alerta de presupuesto se modela como efecto de “Registrar gasto” y se consume en “Ver notificaciones”. La Fig. 1b muestra además que la gestión de miembros y la emisión de invitaciones comparten al administrador del hogar como actor principal.

V-B. Diagrama de componentes

La Fig. 2 descompone el sistema en los componentes de presentación, rutas, controladores, servicios, modelos y persistencia. La Fig. 2 evidencia que las dependencias apuntan hacia abajo: las vistas y las rutas conocen a los controladores, los controladores a los servicios y los servicios a los modelos; ningún componente inferior importa



(a) Operaciones financieras

(b) Hogar y usuarios

Fig. 1: Diagramas de casos de uso del sistema

a uno superior. Esa dirección es la misma que impuso el orden de migración por capas descrito en la Sección III.

V-C. Diagrama de secuencia

La Fig. 3 describe el registro de un gasto desde el envío del formulario hasta la redirección final. La Fig. 3 evidencia que la validación de sesión y hogar ocurre antes

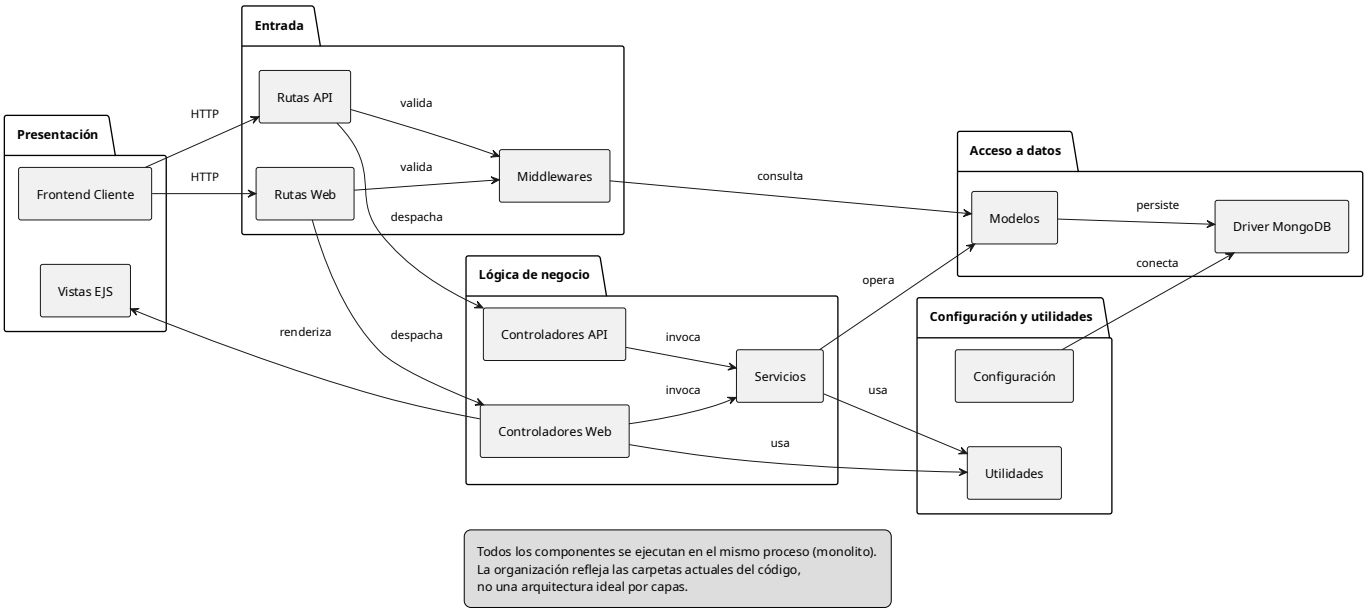


Fig. 2: Componentes y dirección de dependencias

de invocar al controlador, que el servicio concentra la creación y la evaluación del presupuesto, y que la respuesta al usuario no espera a que la notificación se consulte. La secuencia condensa retornos triviales para mejorar la legibilidad; el flujo funcional es equivalente al original de 22 mensajes.

V-D. Diagrama de colaboración

La Fig. 4 presenta la misma operación como una red de objetos enlazados por mensajes numerados. La Fig. 4 evidencia el anidamiento de las llamadas: el mensaje 3.1.2.1 representa la persistencia en MongoDB y cubre tanto las consultas como las escrituras que el modelo resuelve. Los ocho objetos incluyen la base de datos, de modo que la colaboración no oculta el extremo de persistencia.

V-E. Diagramas de clases

V-E1. Dominio financiero: La Fig. 5a reúne las entidades de cuenta, categoría, transacción, presupuesto, meta y aporte. La Fig. 5a evidencia que las transacciones referencian una cuenta y una categoría, que los presupuestos limitan una categoría por mes y que los aportes se componen dentro de la meta. Las entidades se delimitan por el hogar y conservan los montos como enteros en la unidad mínima, decisión que evita errores de punto flotante.

V-E2. Usuarios y hogar: La Fig. 5b presenta las entidades de usuario, hogar, invitación y notificación. La Fig. 5b evidencia que la membresía se modela como un subdocumento del hogar con un rol tipado, que las invitaciones relacionan al usuario que las crea con el hogar que las emite y que las notificaciones pertenecen a un usuario y a un hogar. Las enumeraciones de rol, estado de invitación y tipo de notificación restringen los valores válidos sin cambiar los datos almacenados.

Tabla V: Resultados al cierre de la migración (commit `34016bd`)

Indicador	Antes	Después
Errores de `tsc --noEmit`	686	0
Comprobaciones HTTP exitosas	—	204/204
Archivos `.mjs` en `src/`	53	0
Referencias a `.mjs` en el código	106	0

Fuente: suites HTTP ejecutadas sobre el commit `34016bd` (05/11/2026) y `npm run typecheck` sobre el mismo commit.

VI. Resultados, validación y lecciones aprendidas

VI-A. Resultados de la migración

Al cierre de la migración, el compilador dejó de emitir diagnósticos sobre el árbol completo y las suites HTTP quedaron sin fallos. La Tabla V reúne únicamente las cifras de ese momento; no incorpora el conteo de comprobaciones del estado actual, que se discute en la Sección VI-D. El descenso de 686 a 0 errores de `tsc --noEmit` se produjo por capas y sin cambios de comportamiento observable, y las 204 comprobaciones funcionales se mantuvieron en verde tras cada cierre de capa.

VI-B. Validación funcional

La validación combinó dos niveles. El primero fue estático: `npm run typecheck` ejecuta `tsc --noEmit` sobre `src/` y exige cero diagnósticos. El segundo fue funcional: las suites de HTTP se ejecutaron contra el servidor iniciado con `tsx` y la base MongoDB activa. La suite de estado base preparó usuarios, hogares y sesiones; las suites de transacciones, presupuestos y permisos, reportes y recursos, y autenticación e interfaz de programación aportaron

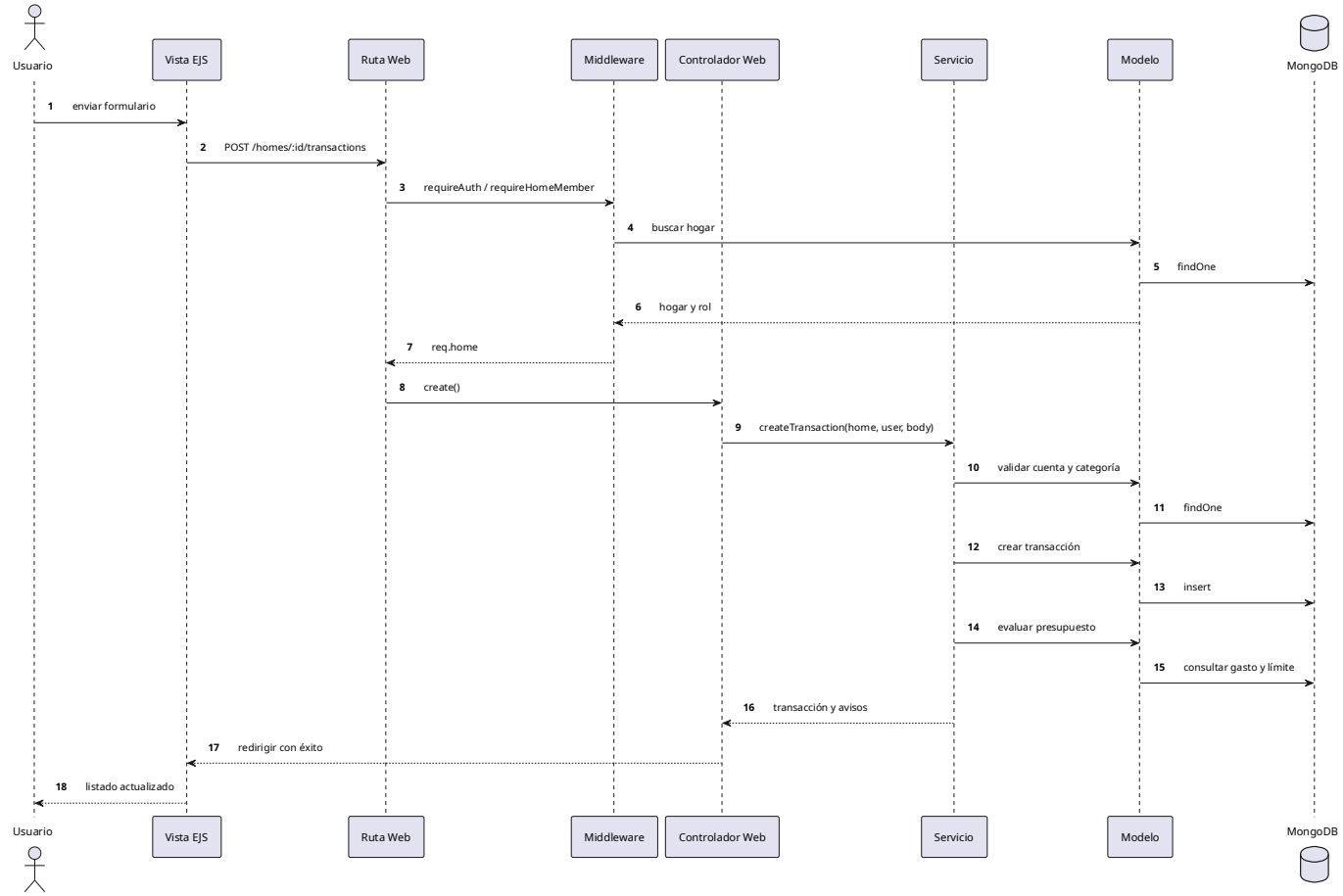


Fig. 3: Secuencia de registro de un gasto.

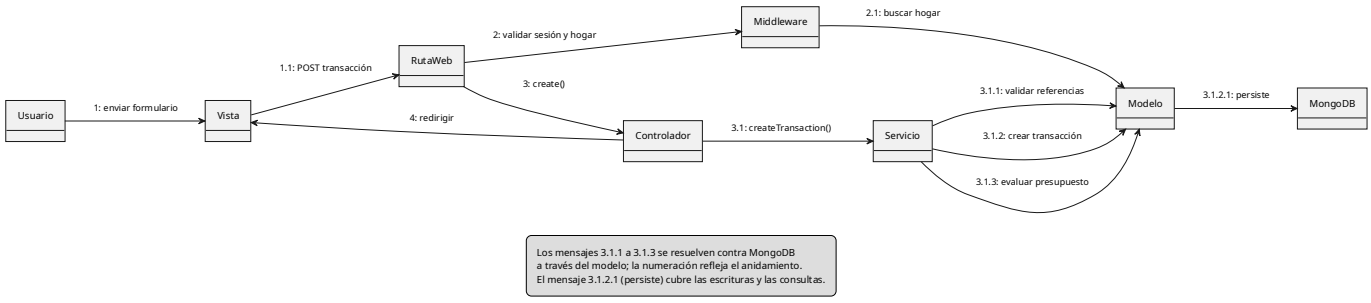
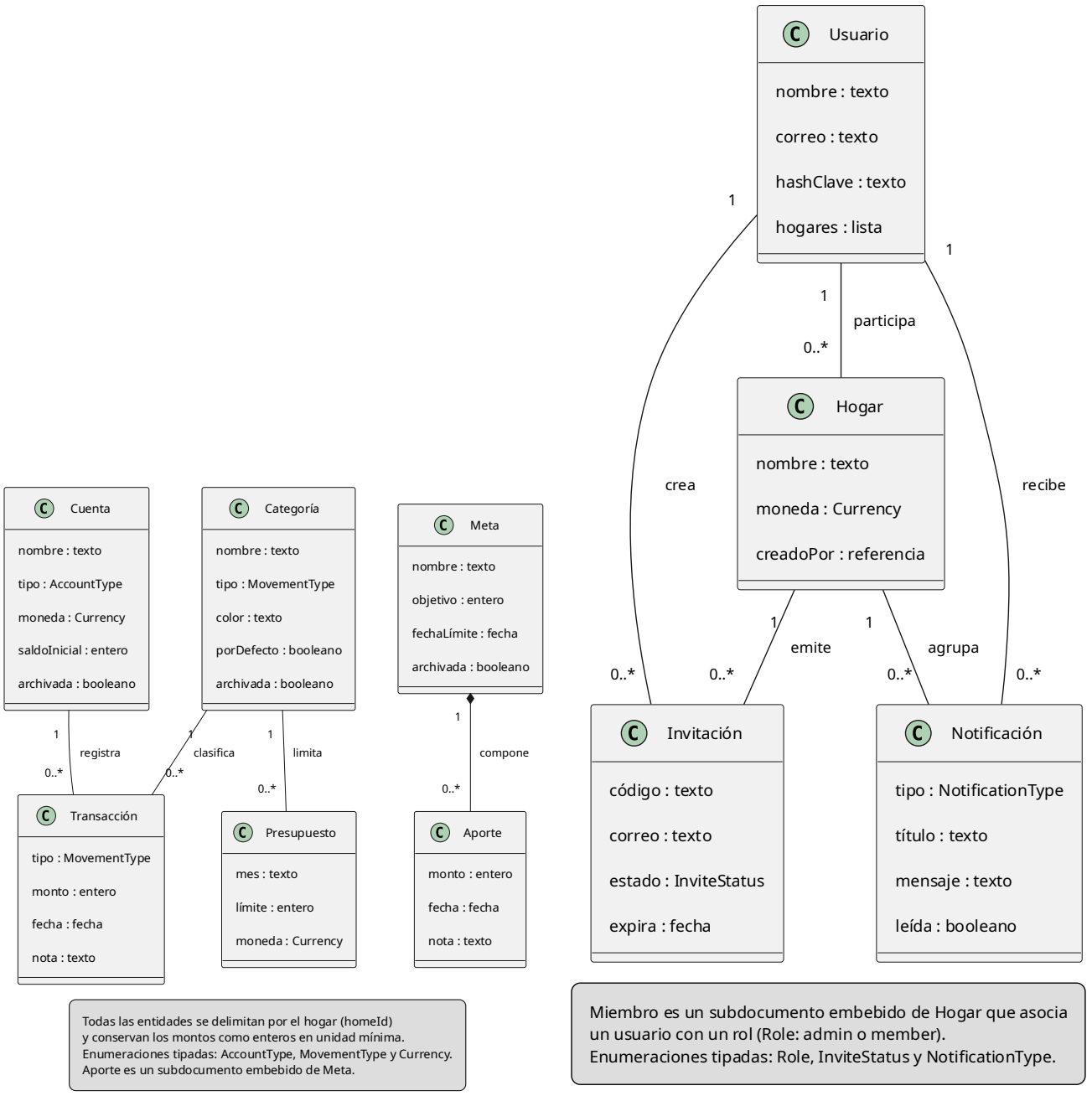


Fig. 4: Colaboración de objetos en el registro de un gasto.

52, 62, 49 y 41 comprobaciones, respectivamente, hasta sumar 204.

A ello se añadieron comprobaciones puntuales: el documento OpenAPI respondió con el HTML de Swagger y su archivo de inicialización; las rutas `/js/app.js` y `/js/dashboard.js` sirvieron los paquetes generados; y el mensaje de sesión se conservó con una sola aparición. La ausencia de residuos se verificó con dos búsquedas: ninguna extensión `.mjs` permaneció bajo `src/` y ninguna referencia a esa extensión sobrevivió en `src/` o `public/`. Las suites son scripts de shell alojados en `.data/tests`; no forman parte de los scripts de `package.json` y requieren MongoDB y el servidor en ejecución, por lo que su reproducción completa depende de ese entorno.



(a) Dominio financiero

(b) Usuarios y hogar

Fig. 5: Diagramas de clases del dominio

VI-C. Evidencia de snapshots

La comparación se apoya en ocho pares de archivos reconstruidos desde el historial: el punto de entrada, el modelo de usuario, el modelo de invitación, el servicio de cuentas, el middleware de hogar, el controlador de cuentas de la interfaz de programación, el controlador web de autenticación y la fuente del frontend. El par del modelo de usuario, reproducido en el Listado 2, es representativo: muestra la interfaz del documento, la interfaz del contrato y la clase cuyos estáticos se copian con `loadClass`. En conjunto, los ocho pares muestran que la migración conservó la estructura de cada archivo y añadió tipos en los contratos, los parámetros y los retornos, sin alterar los flujos.

VI-D. Doble alcance temporal

Este artículo distingue dos momentos: el proceso migratorio (cifras 686 a 0 y 204/204, commit `34016bd`) y el estado actual del repositorio (67 archivos `.ts`, 577/577

checks en `cebc62a`). Las cifras actuales no se contabilizan como resultados de la migración porque corresponden a las fases A–E posteriores. La distinción es una decisión metodológica: evita atribuir al proceso migratorio funciones y pruebas que se incorporaron después de su cierre y permite comparar ambos momentos sin mezclar sus fuentes.

VI-E. Despliegue

El despliegue en Vercel no ejecuta un paso de compilación: el servidor se inicia con `tsx` y el sitio sirve los artefactos ya publicados en `public/`. Esa configuración mantiene el typecheck como una verificación previa y no como un requisito del arranque. La lección de despliegue fue la resolución de tipos: Vercel instala dependencias con un anidamiento distinto al local, por lo que `tsconfig.json` declara `typeRoots` con las rutas `./node_modules/@types`, `../node_modules/@types` y `../../node_modules/@types`. Sin esa declaración, el paquete de tipos de Node.js no se encontraba en el entorno de ejecución remoto.

VI-F. Lecciones aprendidas

Tres lecciones resumen el proceso. Primera: en Mongoose, extender una clase desde `Model` no aporta tipado porque `Model` es una interfaz; el contrato debe declararse como interfaz y los estáticos implementarse en una clase copiada con `loadClass`. Segunda: activar `strict` de forma incremental por capas convierte el conteo de errores en una señal de avance y evita mezclar deuda localizada con ruido cruzado. Tercera: separar el typecheck del build permite verificar el árbol sin generar artefactos y conserva un despliegue sin paso de compilación.

VII. Conclusiones

VII-A. Síntesis técnica

La migración transformó un monolito JavaScript ESM en un árbol TypeScript estricto sin reescribir la aplicación. El renombrado mecánico, la configuración del compilador, el orden por capas y la formalización de los modelos permitieron llegar a cero diagnósticos y mantener 204 comprobaciones funcionales. El patrón de interfaz de documento, interfaz de modelo y clase de estáticos resolvió el caso de Mongoose y se generalizó a los nueve modelos actuales.

VII-B. Cumplimiento de objetivos

Los objetivos se cumplieron con evidencia reproducible: el typecheck desciende de 686 a 0, las suites cierran en 204/204 y cada par de snapshots queda anclado a un commit. El valor académico no reside solo en el resultado, sino en el protocolo: cualquier lector puede repetir el typecheck, ejecutar las suites en un entorno equivalente y comparar los archivos representativos entre `fb1ad1d` y `34016bd`. La documentación de las decisiones y de las alternativas descartadas completa esa trazabilidad.

VII-C. Trabajo futuro

Quedan tres líneas abiertas: incorporar pruebas unitarias sobre servicios y modelos, añadir una integración continua que ejecute el typecheck en cada cambio, y versionar el contrato OpenAPI junto con el código para detectar divergencias entre la documentación y los manejadores. Ninguna de esas líneas modifica las conclusiones de este artículo.

Tabla VI: Alternativas descartadas durante la migración

Alternativa	Motivo de descarte	Opción adoptada	Evidencia
`class XModel extends Model`	`Model` es interfaz; colapsa miembros a `any`	`IXModel` + clase con `loadClass`	`models/user.ts`
`tsc` + ejecución de `dist/`	Añade build al despliegue; Vercel no lo ejecuta	`tsx` en runtime + `tsc --noEmit`	`package.json`
`catch (err: any)`	Anula el chequeo estricto del error	`unknown` + `isHttpError`	`middlewares/ error.ts`
Guardia de hogar por ruta	Duplica control y altera flujos de error	Aserción no nula tras el middleware	`types/ express.d.ts`
Frontend sin migrar	Mantiene dos lenguajes y contratos distintos	TypeScript + esbuild con artefactos versionados	`package.json` (`build:ui`)

Fuente: repositorio en el commit `34016bd` (05/11/2026).

Referencias

[1] OpenAPI Initiative, "OpenAPI Specification 3.1.0," https://swagger.io/specification/, 2026, consultado el 9 de noviembre de 2026.

[2] privatenumber, "tsx: Typescript execute," https://github.com/privatenumber/tsx, 2026, consultado el 9 de noviembre de 2026.

[3] Microsoft, "Typescript handbook," https://www.typescriptlang.org/docs/handbook/intro.html, 2026, consultado el 9 de noviembre de 2026.

[4] ——, "TSConfig Reference: strict," https://www.typescriptlang.org/tsconfig/strict.html, 2026, consultado el 9 de noviembre de 2026.

[5] Mongoose, "Mongoose API: Model," https://mongoosejs.com/docs/api/model.html, 2026, consultado el 9 de noviembre de 2026.

[6] E. Wallace, "esbuild: an extremely fast bundler for the web," https://esbuild.github.io/, 2026, consultado el 9 de noviembre de 2026.

[7] OpenJS Foundation, "Express: fast, unopinionated, minimalist web framework for node.js," https://expressjs.com/, 2026, consultado el 9 de noviembre de 2026.

[8] Object Management Group, "Unified modeling language, version 2.5.1," https://www.omg.org/spec/UML/2.5.1/, 2026, consultado el 9 de noviembre de 2026.

[9] PlantUML, "Plantuml language reference guide," https://plantuml.com/guide, 2026, consultado el 9 de noviembre de 2026.

Anexo A

A.1 Decisiones y alternativas descartadas

La Tabla VI resume las cinco alternativas evaluadas durante la migración. Todas fueron descartadas por un motivo verificable y sustituidas por la opción que quedó en el repositorio.

A.2 Trazabilidad de snapshots

Los ocho pares documentados en la Tabla VII se utilizaron como evidencia del efecto de la migración; los commits indican el estado previo y el cierre de la migración. El estado actual del árbol puede diferir por las fases A–E posteriores.

Tabla VII: Pares de archivos comparados entre el estado previo y el cierre de la migración

Par	Archivo antes	Archivo después	Commit antes	Commit después
Entrada	`app.mjs`	`app.ts`	`fb1ad1d`	`34016bd`
Usuario	`user.mjs`	`user.ts`	`fb1ad1d`	`34016bd`
Invitación	`invite.mjs`	`invite.ts`	`fb1ad1d`	`34016bd`
Cuentas	`accountService.mjs`	`accountService.ts`	`fb1ad1d`	`34016bd`
Hogar	`home.mjs`	`home.ts`	`fb1ad1d`	`34016bd`
API cuentas	`accountController .mjs`	`accountController .ts`	`fb1ad1d`	`34016bd`
Web auth	`authController.mjs`	`authController.ts`	`fb1ad1d`	`34016bd`
Frontend	`public/js/app.js`	`src/frontend/app.ts`	`fb1ad1d`	`34016bd`

Fuente: historial del proyecto; `fb1ad1d` identifica el estado previo y `34016bd` el cierre de la migración (05/11/2026).

Los ocho pares abarcan la entrada, el dominio, los servicios, los controladores y el frontend, de modo que la evidencia no se limita a un módulo aislado. En todos los casos se conservaron las rutas y los flujos, y los cambios se concentraron en las firmas, los contratos y las anotaciones de tipo. Esa continuidad permite atribuir el resultado al proceso migratorio y no a una reescritura.

Anexo B

B. Casos de uso específicos

Los casos de uso específicos del sistema, con actores, precondiciones y flujos detallados, se conservan en el documento original del proyecto. La lógica funcional no cambió durante la migración, por lo que esos casos siguen siendo válidos para el sistema migrado. El documento original está disponible en https://github.com/JuancMerchanM/Proyecto_GestionFinanciera/blob/c55ce3e8b4d56953ed697f155e9f3fce0dbe98d0/CasosUso.pdf.

Anexo D

D. Repositorio del proyecto

El código fuente completo del proyecto migrado, junto con el PDF de esta documentación, está disponible en el repositorio público: https://github.com/JuancMerchanM/Proyecto_GestionFinanciera.